

Predictive Modeling for Course Demand and Revenue Forecasting on EduPro

Research Paper

Tejpratap Prajapati

September 2026

Abstract

Online learning platforms sit on a lot of data that mostly gets used to look backward — how many people enrolled, how much a course made, which instructors did well. This project tries to look forward instead. Working with EduPro's course, teacher, and enrollment data, I built a small predictive pipeline that estimates two things for any given course: how many people are likely to enroll, and how much revenue it's likely to generate.

The work moved through the usual stages — cleaning the raw sheets, merging course and teacher data, running exploratory analysis, engineering features, splitting the data, testing a baseline, then trying progressively more capable models. Nine features went into the models in the end: four categorical ones (CourseCategory, CourseType, CourseLevel, Expertise) and five numeric ones (CoursePrice, CourseDuration, CourseRating, TeacherRating, YearsOfExperience). EnrollmentCount and CourseRevenue were the two targets.

I evaluated everything on MAE, RMSE, and R^2 , and honestly, the two problems turned out to behave very differently — revenue responded well to a simple linear model, while enrollment did not. Those results, along with the reasoning behind picking a final model for each target, are covered in detail below. Everything was eventually wrapped into a Streamlit app so the numbers aren't just sitting in a notebook — a user can plug in course details and get a prediction along with some plain-language interpretation.

Keywords: Predictive Modeling, Course Demand Forecasting, Revenue Forecasting, Machine Learning, Regression, Feature Engineering, EduPro, Streamlit, Business Analytics

1. Introduction

Every online course platform accumulates a huge trail of data — who took what, how they rated it, what the instructor's background was, how much money came in. It's easy to build dashboards that describe what already happened. It's a different, harder problem to estimate what will happen with a course that hasn't launched yet, or to compare two possible course ideas before committing resources to either.

That's the gap this project tries to close for EduPro. Instead of only reporting historical numbers, I wanted to see whether the platform's existing course and instructor characteristics could actually predict enrollment and revenue with any reliability.

The project followed a fairly standard data science arc: clean the data, explore it, engineer features, train and evaluate models, then package the result into something usable. What made it worthwhile, in my view, wasn't just producing two trained models — it was turning them into a dashboard that a non-technical person at EduPro could actually open and use.

2. Problem Statement

EduPro doesn't currently have a systematic way to estimate how a course will perform before or shortly after it launches. Decisions about pricing, which instructors to bring on, and which categories to expand tend to rely on gut feeling or on looking at similar past courses by eye.

This project sets out to replace some of that guesswork with two concrete prediction tasks.

2.1 Course Demand Prediction

Given a course's characteristics, estimate its likely **EnrollmentCount**.

2.2 Course Revenue Prediction

Using the same characteristics, estimate the course's likely **CourseRevenue**.

Neither prediction is meant to be treated as gospel — they're inputs into a decision, not a substitute for one.

3. Project Objectives

Broadly, I set out to:

1. Clean and prepare the EduPro dataset.
2. Understand patterns in course, teacher, enrollment, and revenue data.
3. Narrow down which features actually matter for prediction.
4. Build a clean prediction-target dataset.
5. Train a model for course demand.
6. Train a model for course revenue.
7. Compare a naive baseline against real machine-learning models.
8. Judge performance using MAE, RMSE, and R^2 rather than a single number.
9. Package the winning models into a reusable pipeline.
10. Put that pipeline behind a Streamlit interface.
11. Translate the model outputs into recommendations a business person could act on.
12. Stress-test the dashboard so it doesn't break on bad input.

4. Dataset and Data Sources

The source file behind all of this is **EduPro Online Platform(1).xlsx**, which is really three separate sheets stitched together for different purposes.

4.1 Users Dataset

Roughly 3,000 user records, with five fields: UserID, UserName, Age, Gender, and Email. This sheet mostly sat in the background — it wasn't central to the modeling, but it rounded out the picture of who's on the platform.

4.2 Teachers Dataset

60 instructor records across seven columns, covering identity, demographics, subject expertise, years of experience, and a teacher rating. Fields worth calling out: TeacherID, TeacherName, Age, Gender, Expertise, YearsOfExperience, TeacherRating.

4.3 Courses Dataset

Also 60 records — one per course — describing the characteristics that matter for prediction. I merged this with the relevant teacher fields so each course row carried both its own attributes and its instructor's attributes.

5. Data Preparation and Cleaning

Before touching any models, I went through the standard checks: missing values, duplicate rows, columns with inconsistent types, anything that looked off. I kept the original sheets untouched and worked from copies throughout, mainly so I could always go back and re-check something against the source if a number looked strange later on.

Merging the course and teacher data into a single modeling table was the bulk of the prep work. The result was saved as **EduPro_Day8_Prediction_Targets.xlsx**, in a sheet called **Prediction_Targets**.

That final table came out to **60 rows and 15 columns**, with no missing values and no duplicate CourseIDs — a reasonably clean starting point, though small.

6. Modeling Dataset

The Day 8 table carried these fields: CourseID, CourseName, CourseCategory, CourseType, CourseLevel, CoursePrice, CourseDuration, CourseRating, TeacherID, TeacherName, TeacherRating, YearsOfExperience, Expertise, EnrollmentCount, CourseRevenue.

Of these, EnrollmentCount and CourseRevenue became the two prediction targets.

7. Feature Selection

I ended up using nine of the available fields as model inputs.

7.1 Categorical Features

CourseCategory, CourseType, CourseLevel, and Expertise. All four went through one-hot encoding as part of the preprocessing pipeline, since none of them are ordinal in any meaningful way.

7.2 Numerical Features

CoursePrice, CourseDuration, CourseRating, TeacherRating, and YearsOfExperience — these went in as-is, no transformation needed at this stage.

7.3 What I Left Out

CourseID, CourseName, TeacherID, and TeacherName were dropped from training. They identify a row rather than describe it, so keeping them in would just add noise (or worse, let the model memorize specific courses instead of learning general patterns).

8. Exploratory Data Analysis

Before jumping into modeling, I spent a fair amount of time just looking at the data — category distributions, pricing spreads, duration, ratings on both the course and teacher side, and how enrollment and revenue varied across categories.

Some of the things I looked at:

- How courses split across categories and types
- Distribution of course levels
- Pricing and duration ranges
- Course ratings versus teacher ratings
- Enrollment and revenue patterns
- How performance differed by category
- Whether teacher experience correlated with course outcomes

None of this fed directly into a chart in this document, but it shaped which features I trusted going into modeling and gave me an early sense that revenue and enrollment weren't going to behave the same way.

9. Prediction Target Analysis

9.1 EnrollmentCount

Statistic	Value
Mean	166.6667
Standard Deviation	12.5234
Minimum	140
Maximum	196

No course had zero enrollment, and the spread here is fairly tight — the highest-enrolling course isn't dramatically ahead of the lowest. That narrow range turned out to matter a lot later, when the models struggled to find meaningful signal in such a compressed target.

9.2 CourseRevenue

Statistic	Value
Mean	15188.7245
Standard Deviation	25409.7219
Minimum	0
Maximum	85416.6

Revenue is a completely different story — 38 of the 60 courses had zero recorded revenue, and the ones that did make money varied enormously. That combination of a lot of zeros plus a long tail of high performers is exactly the kind of distribution that can make a target either very easy or very hard to model, depending on what's driving it.

10. Train-Test Strategy

With only 60 rows total, I split 80:20 rather than holding out a larger test set — anything more aggressive would have left too little to train on.

Dataset	Observations
Training	48
Testing	12
Total	60

The split was saved as **EduPro_Day9_Train_Test_Splits.xlsx**, containing Train_Data, Test_Data, X_Train, X_Test, Y_Train, and Y_Test as separate sheets, so the same split could be reused consistently across every model I tried afterward.

11. Baseline Modeling

Before trusting any real model, I wanted a sanity-check baseline — just predicting the mean of the training target every time — to know what “doing nothing clever” looks like.

11.1 Enrollment Baseline

Metric	Result
MAE	10.4444
RMSE	11.7745
R^2	-0.0472

11.2 Revenue Baseline

Metric	Result
MAE	28413.5568
RMSE	36217.3447
R^2	-0.2488

Both baselines came out with negative R^2 , which just confirms that predicting a flat average doesn't explain much variation in either target — not a surprise, but useful to have as a floor.

12. Linear Regression

12.1 Enrollment Prediction

Metric	Result
MAE	14.2038
RMSE	19.8125
R^2	-1.9650

This one caught me off guard a little — Linear Regression actually did worse than just guessing the mean. An R^2 this negative means the model's errors are considerably larger than the baseline's, which tells me the relationship between these nine features and enrollment isn't linear, or the signal simply isn't strong enough given how tight the enrollment range is.

12.2 Revenue Prediction

Metric	Result
MAE	2725.9923
RMSE	3542.4712
R^2	0.9881

Revenue went the opposite direction — a huge jump over the baseline, and an R^2 close to 1. I'll flag now, and again later in the limitations section, that a result this strong on only 12 test rows

deserves some skepticism rather than pure celebration; it's worth re-checking for any feature that might be too closely tied to how revenue was calculated in the first place.

13. Advanced Modeling

Given how oddly Linear Regression behaved on enrollment, I brought in a couple of tree-based models to see if they could pick up nonlinear patterns that a straight line couldn't.

Random Forest Regressor

Configured with `n_estimators = 200`, `random_state = 42`, `max_depth = None`, `min_samples_split = 2`, `min_samples_leaf = 1`.

Gradient Boosting Regressor

Configured with `n_estimators = 100`, `learning_rate = 0.05`, `max_depth = 2`, `random_state = 42`.

The hope was that either of these could capture interactions between, say, price and category, or rating and experience, that a linear model would miss entirely.

14. Model Evaluation

I leaned on three metrics rather than picking one favorite, since each tells you something different.

14.1 Mean Absolute Error

The average size of the error, in the same units as the target — easy to explain to someone who isn't a data person.

14.2 Root Mean Squared Error

Similar to MAE but penalizes big misses more heavily, which matters if a handful of large errors would be more damaging than many small ones.

14.3 R^2

The share of variance the model actually explains, with 1.0 being a perfect fit and negative values meaning the model is worse than just guessing the average.

Looking at all three together avoided the trap of picking a model that looks good on one metric but is quietly bad on another.

15. Final Model Selection

Target	Final Model	MAE	RMSE	R^2
EnrollmentCount	Random Forest	12.1971	13.2711	-0.3303

Target	Final Model	MAE	RMSE	R ²
CourseRevenue	Linear Regression	2725.9923	3542.4712	0.9881

Random Forest was the best of the enrollment models I tried, but I want to be upfront that “best available” here still isn’t “good” — a negative R² means it’s still underperforming a flat average prediction. I kept it as the final model because it was the strongest option on hand, not because it met a quality bar I’d consider production-ready. Linear Regression, on the other hand, earned its spot on revenue purely on merit. These are the two models that actually feed the pipeline and the dashboard.

16. Feature Engineering and Preprocessing

The categorical fields went through one-hot encoding, specifically **OneHotEncoder(handle_unknown=“ignore”)**. That “ignore unknown” setting matters more than it might seem — once this is deployed, someone could type in a course category that never appeared in training, and without that setting the whole prediction call would just throw an error instead of degrading gracefully.

Both the encoder and the models were wrapped into scikit-learn pipelines, so the exact same transformations happen whether you’re training or making a live prediction — no risk of the two drifting apart over time.

17. Final Prediction Pipeline

At a high level, a new course’s details flow through: input validation, feature preparation, preprocessing, and then into the enrollment model and revenue model in parallel, each producing its own prediction. Keeping the two models running side by side off the same preprocessed input made the pipeline simpler to maintain than having two separate, diverging code paths.

18. Streamlit Dashboard

Everything above eventually lives inside a Streamlit app, with the main file at `**D:Analytics Project_Predictive_Modeling.py**`. It grew over several days of work (roughly Days 15 through 19), starting simple and gradually picking up more functionality.

A user fills in course category, type, level, price, duration, course rating, teacher rating, years of experience, and expertise, and the app returns predictions for both enrollment and revenue.

19. Demand Prediction Dashboard

Rather than just spitting out a raw number, the demand section buckets the predicted enrollment into a rough category — low, moderate, strong demand, that sort of thing — and

pairs it with a short recommendation. The goal was to save a business user from having to interpret “predicted enrollment: 158” on their own.

20. Revenue Prediction Dashboard

Same idea, applied to revenue — the dashboard shows the predicted number and frames it in terms someone could actually use: is this course likely to be a strong earner, worth a closer look on pricing, worth prioritizing over another idea, and so on.

21. Category Analysis

Added on Day 18, this section zooms out from a single course to look at category-level performance as a whole. It’s meant to help someone ask which categories are consistently doing well, which ones might need attention, and where there might be room to add more courses. I’d treat these observations as a starting point for a conversation, not a final verdict — with only 60 courses spread across several categories, some categories don’t have many data points behind them.

22. Feature Importance Analysis

Also from Day 18 — a ranked list (and a chart) of which features the tree-based model is leaning on most heavily. It’s a genuinely useful diagnostic, but I want to be careful not to oversell it: feature importance tells you what the model found useful for prediction, not necessarily what causes enrollment or revenue to go up in the real world.

23. Dashboard Testing and Error Handling

Day 19 was mostly about breaking the app on purpose to see what would happen, then fixing it.

Valid Inputs

Confirmed the happy path works — reasonable inputs produce a reasonable prediction.

Missing Inputs

The app needs to say something useful rather than just failing silently or returning garbage.

Invalid Numerical Values

Things like negative prices or absurd durations get caught by validation instead of being fed straight into the model.

Unknown Categorical Values

This is where **handle_unknown="ignore"** from the preprocessing step earns its keep — an unfamiliar category doesn't crash the app.

Missing Model Files

If the saved model files aren't where the app expects them, it should say so clearly instead of throwing a raw stack trace at the user.

Prediction Errors

Any failure during the actual prediction call gets caught and turned into a readable message rather than an ugly traceback.

24. Results and Findings

A few things stood out by the end of this project.

Enrollment and revenue are just not the same kind of problem. Enrollment sits in a narrow band and turned out to be genuinely hard to predict well with the features available — even the best model I tried still underperforms a flat average. Revenue, on the other hand, has a much wider spread (including a lot of zeros) and responded very well to a simple linear model — almost too well, which is a point I keep coming back to in the limitations section.

The baseline comparison was worth doing if only to prove that “just guess the average” isn't good enough for either target, and to make the later improvements (or lack thereof, in enrollment's case) meaningful rather than assumed.

Trying tree-based models on top of Linear Regression was the right call — it gave enrollment at least some improvement over a straight linear fit, even if the end result is still not strong in absolute terms.

By the end, the pipeline and dashboard turned what would otherwise be a one-off analysis into something reusable — plug in a new course's details, get both predictions back, with plain-language interpretation attached.

25. Business Recommendations

25.1 Course Launch Planning

Predicted enrollment can be one input among several when deciding whether a new course idea is worth greenlighting — just not the only input, especially given how weak the enrollment model currently is.

25.2 Pricing Decisions

Revenue predictions can inform pricing conversations, but should sit alongside competitor pricing, perceived course quality, and market demand rather than replacing that judgment.

25.3 Instructor Planning

Teacher rating, experience, and area of expertise are already baked into the model, so they're a natural angle for thinking through which instructor might be a good fit for a new course.

25.4 Category Strategy

Category-level patterns can help flag where EduPro might want to invest more, though I'd want a larger sample before treating any single category's numbers as conclusive.

25.5 Data-Driven Decision Making

None of this replaces human judgment — it's meant to sit alongside it, giving people a number to react to and question rather than a decision handed down automatically.

26. Limitations

26.1 Dataset Size

Sixty courses is a small dataset for any kind of machine learning, and the 48/12 train-test split makes the test set particularly small. Results — good or bad — should be read with that in mind.

26.2 Limited Historical Information

There's likely a lot influencing enrollment and revenue in the real world (marketing spend, timing, competitor courses, learner demand cycles) that simply isn't captured in this dataset.

26.3 Revenue Distribution

The mix of many zero-revenue courses and a long tail of high performers makes revenue an inherently trickier target than it might look from the R^2 alone.

26.4 Prediction Uncertainty

These are statistical estimates, not guarantees — a course predicted to do well can still underperform, and vice versa.

26.5 Feature Availability

The models are only as good as the nine features feeding them; if a genuinely important driver of demand isn't in the dataset, no amount of modeling will surface it.

27. Future Scope

27.1 Larger Dataset

More historical courses would help every part of this analysis, especially the enrollment model.

27.2 Time-Based Features

Launch date, monthly enrollment trends, seasonality, promotional periods, and engagement over time could all add signal that's currently missing.

27.3 Advanced Modeling

Worth revisiting once there's more data — some algorithms only start to shine past a certain dataset size.

27.4 Hyperparameter Optimization

I kept the model configurations fairly standard here; a proper tuning pass could squeeze out more performance, particularly for enrollment.

27.5 Model Monitoring

Once deployed, actual outcomes should be tracked against predictions so it's obvious if the models start drifting.

27.6 Automated Retraining

Eventually, the models could retrain themselves as new course data comes in, rather than needing a manual refresh.

27.7 Advanced Dashboard Analytics

Confidence intervals, historical-versus-predicted comparisons, trend views, scenario testing, and pricing simulations would all make the dashboard more useful over time.

28. Conclusion

This project took EduPro's course and instructor data through a full pipeline — cleaning, exploration, feature engineering, modeling, evaluation, and finally deployment into a working dashboard. The two targets, EnrollmentCount and CourseRevenue, turned out to need very different treatment: revenue responded well to a simple model, enrollment did not, even after trying more sophisticated approaches.

That gap is, in a way, the most useful finding here — it tells EduPro (and anyone continuing this work) exactly where to focus next: more enrollment-relevant features, a larger dataset, and closer scrutiny of why the revenue model performs as well as it does. The dashboard itself is a solid foundation regardless — it takes course details in and returns interpretable predictions,

category context, and feature-importance insight, all without requiring anyone to open a notebook.

Small dataset aside, I think this sets up a reasonable base for EduPro to build on as more data becomes available.

29. References

1. **Pandas Documentation.** Used throughout for cleaning, merging, and exploring the data.
2. **NumPy Documentation.** Used for numerical operations underlying the analysis.
3. **Scikit-learn Documentation.** Used for the regression models, preprocessing pipelines, train-test splitting, and evaluation metrics.
4. **Streamlit Documentation.** Used to build the interactive dashboard.
5. **EduPro Online Platform Dataset** — *EduPro Online Platform(1).xlsx* — the source data behind the entire project.
6. **EduPro Project Files (Days 1–19).** My own notebooks, processed datasets, model outputs, and dashboard code produced over the course of this project.

30. Project Technology Stack

Python, Pandas, NumPy, Scikit-learn, Joblib, Matplotlib, Jupyter Notebook, Streamlit, and Microsoft Excel — a fairly standard stack for a project of this size, chosen more for familiarity and reliability than anything exotic.